데이터베이스(SQL)

9장. CTE(Common Table Expression)

학습 목표

1) CTE의 개념을 설명할 수 있다.
2) WITH 문의 기본 문법을 사용할 수 있다.
3) 서브쿼리와 CTE의 차이를 설명할 수 있다.
4) 복잡한 SQL을 CTE로 가독성 있게 작성할 수 있다.
5) 여러 개의 CTE를 연결하여 단계별로 분석할 수 있다.
6) JOIN과 함께 CTE를 사용할 수 있다.

공식 문서: WITH 문 (Common Table Expressions)

전체 흐름

1) CTE가 필요한 이유 → 2) CTE의 개념 → 3) 성능에 대한 오해 → 4) WITH 문법 → 5) 서브쿼리와 비교

6) 여러 CTE → 7) 장점과 주의사항

01 CTE의 개념

CTE는 왜 필요할까?

고객별 할인 전 금액을 계산한 후 금액이 10,000보다 큰 고객만 조회하려면 이렇게 작성할 수 있다.

```
SELECT *
FROM (
   SELECT
      customer_id,
      SUM(od.unit_price * od.quantity) AS gross_amount -- 할인 전 금액
   FROM order_details od
   JOIN orders o ON od.order_id = o.order_id
   GROUP BY customer_id
) t
WHERE gross_amount > 10000;
```

결과는 맞지만 SQL이 복잡해질수록 읽기 어려워진다.

서브쿼리의 문제점

1) 여러 단계로 중첩되면 어디부터 읽어야 할지 모른다.

2) 안쪽부터 읽어야 해서 흐름이 거꾸로다.

3) 같은 서브쿼리를 여러 번 써야 할 때 반복된다.

4) 오류가 나면 어느 단계인지 찾기 어렵다.

이를 해결하는 기능이 CTE다.

CTE란?

CTE(Common Table Expression, 공통 테이블 표현식)는 WITH 절을 사용하여 SQL 쿼리 상단에 정의하는 이름이 붙은 임시 결과 집합이다

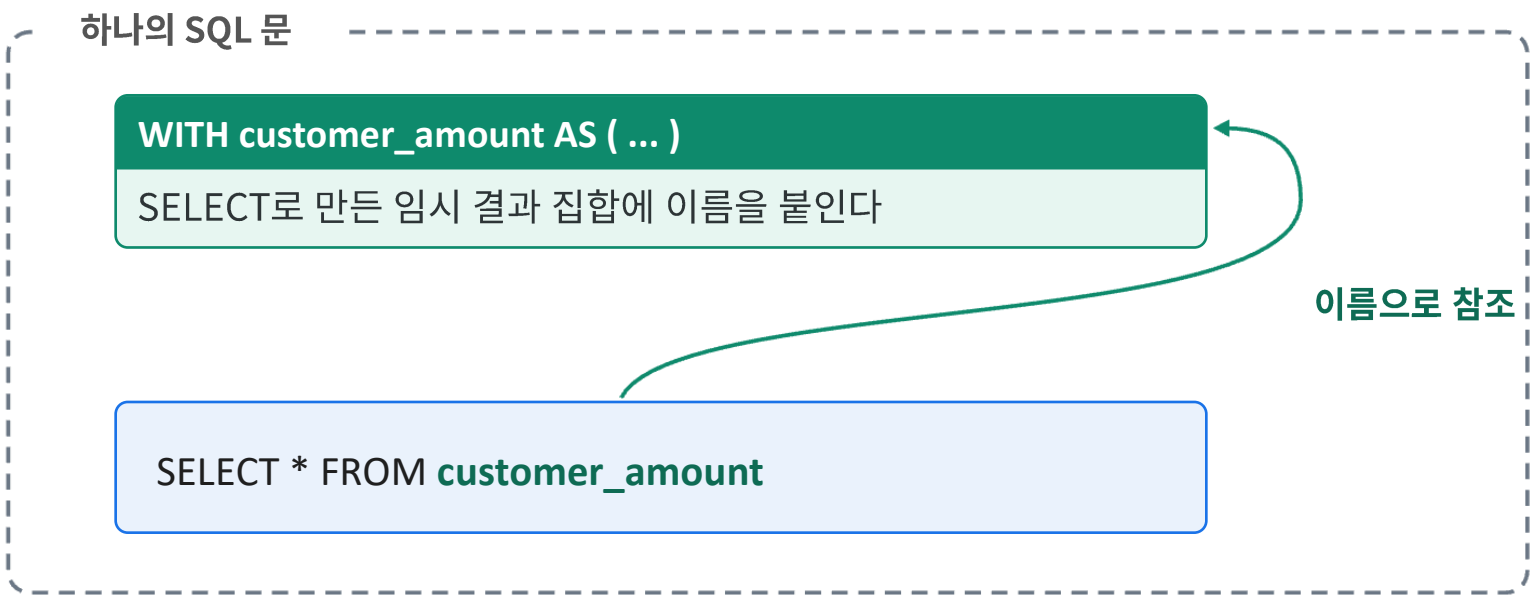
CTE는 CREATE TABLE처럼 **영구적인 데이터베이스 객체를 만들지 않는다.**
CTE 이름은 해당 SQL 문 안에서만 사용할 수 있고 문장이 끝나면 사라진다.

CTE는 파생 테이블을 위에 따로 꺼내 이름을 붙인 형태로 복잡한 서브쿼리에 이름을 붙여서 읽기 쉽게 만드는 기능이다.

공식 문서: WITH Queries

02 WITH 문 기본

WITH 문 기본 구조



문장이 끝나면 이름도 사라진다
(영구 객체를 만들지 않음)

공식 문서: WITH 문법

기본 예제 - Northwind

```
WITH customer_amount AS ( -- 고객별 할인 전 금액
        SELECT
           o.customer_id,
           SUM(od.unit_price * od.quantity) AS gross_amount -- 할인 전 금액
        FROM orders o
        JOIN order_details od ON o.order_id = od.order_id
        GROUP BY o.customer_id
)
SELECT *
FROM customer_amount
ORDER BY gross_amount DESC, customer_id
LIMIT 10;
```

SQL을 읽는 순서

SQL을 읽을 때

1) WITH에서 이름 붙인 계산의 의미를 파악

↓

2) 바깥 쿼리가 그 이름을 어디서 사용하는지 확인

↓

3) 최종 결과의 행 단위와 조건 확인

같은 결과, 다른 작성법 (1/2)

서브쿼리 방식

```
-- 서브쿼리 방식
SELECT *
FROM (
        SELECT
        o.customer_id,
        SUM(od.unit_price * od.quantity) AS gross_amount -- 할인 전 금액
        FROM orders o
        JOIN order_details od ON o.order_id = od.order_id
        GROUP BY o.customer_id
) t
WHERE gross_amount > 10000;
```

같은 결과, 다른 작성법 (2/2)

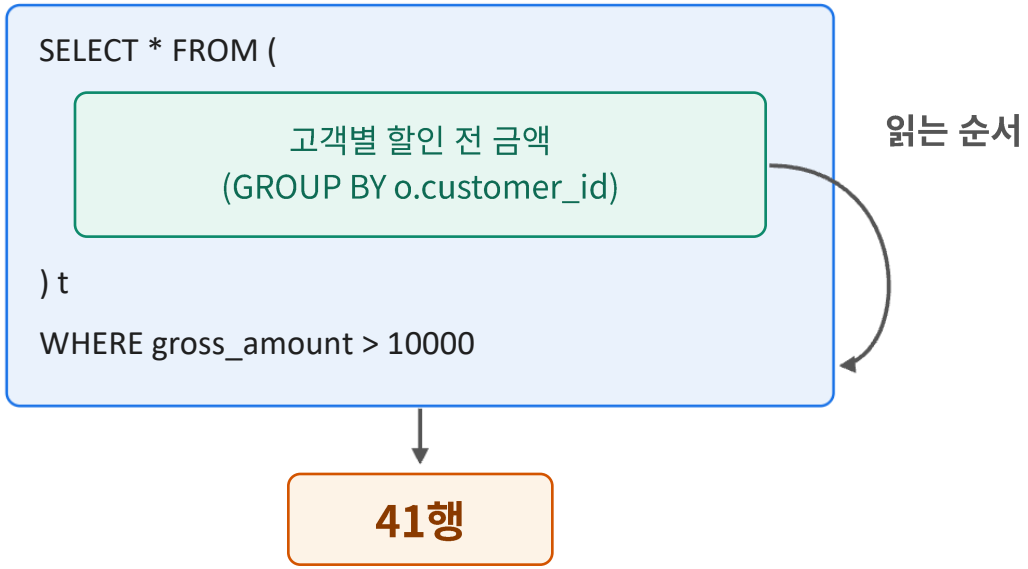
CTE 방식

```
-- CTE 방식
WITH customer_amount AS ( -- 고객별 할인 전 금액
        SELECT
        o.customer_id,
        SUM(od.unit_price * od.quantity) AS gross_amount -- 할인 전 금액
        FROM orders o
        JOIN order_details od ON o.order_id = od.order_id
        GROUP BY o.customer_id
)
SELECT *
FROM customer_amount
WHERE gross_amount > 10000;
```

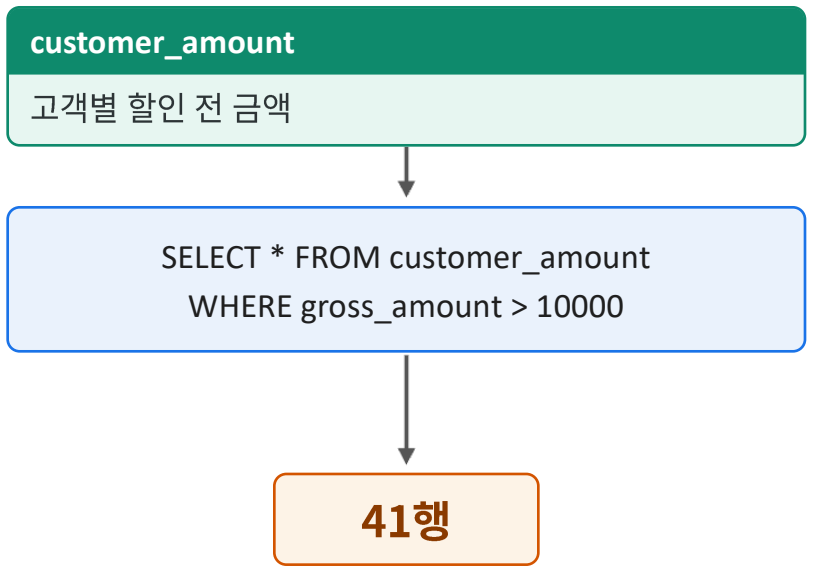
CTE가 위에서 아래로 자연스럽게 읽힌다.

서브쿼리와 CTE의 읽는 방향

서브쿼리: 안쪽부터 읽음



CTE: 위에서 아래로 읽음



03 여러 개의 CTE

여러 개의 CTE 사용하기

CTE는 쉼표로 구분해서 여러 개를 정의할 수 있다.

문법

```
WITH cte1 AS (
...
),
cte2 AS (
...
)
SELECT ...
```

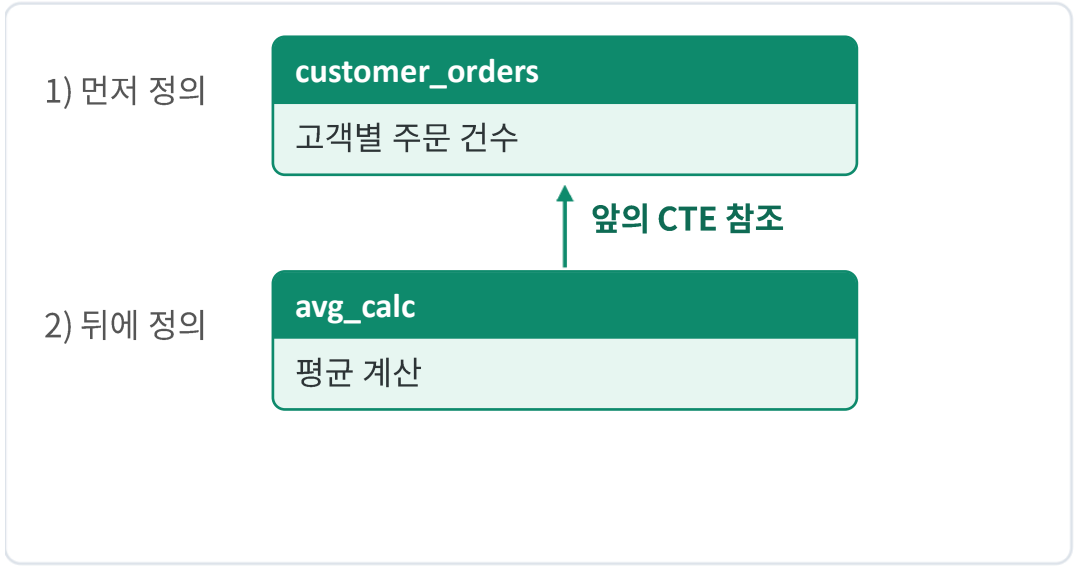
공식 문서: WITH 질의

예제 - 평균 이상 고객 찾기

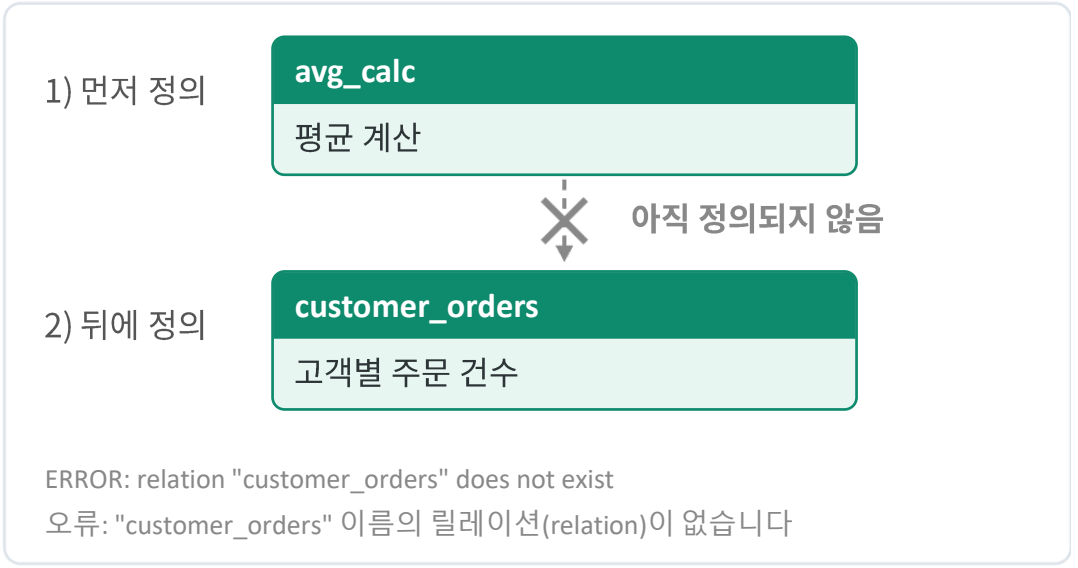
```
WITH customer_amount AS ( -- 고객별 할인 전 금액
        SELECT
          o.customer_id,
          SUM(od.unit_price * od.quantity) AS gross_amount -- 할인 전 금액
        FROM orders o
        JOIN order_details od ON o.order_id = od.order_id
        GROUP BY o.customer_id
),
overall_avg AS ( -- 전체 평균
        SELECT AVG(gross_amount) AS avg_amount -- 고객 평균 금액
        FROM customer_amount
)
```

CTE의 참조 방향

가능



불가능



일반적인 비재귀 WITH에서는 뒤에 정의한 CTE가 앞서 정의한 CTE를 참조한다

CTE 사용 장점

장점	설명
가독성 향상	위에서 아래로 자연스럽게 읽힌다
복잡한 SQL 분리	단계별로 쪼갤 수 있다
재사용	같은 쿼리 안에서 여러 번 참조 가능
유지보수 용이	수정할 부분을 쉽게 찾는다
단계별 디버깅	각 CTE만 따로 실행해서 확인 가능

디버깅

CTE를 만들 때 각 단계를 따로 실행해보면서 확인하면 오류를 빨리 찾을 수 있다.

```
-- 1단계만 먼저 확인
WITH customer_orders AS ( -- 고객별 주문 건수
        SELECT customer_id, COUNT(*) AS order_count -- 주문 건수
        FROM orders GROUP BY customer_id
)
SELECT * FROM customer_orders ORDER BY customer_id LIMIT 10; -- 여기서 결과 확인

-- 확인 후 다음 단계 추가
```

핵심 정리

1) CTE는 Common Table Expression의 약자다.
2) WITH 문으로 작성한다.
3) 임시 결과 집합에 이름을 부여한다.
4) 실제 테이블을 만들지 않으며 SQL 실행 중에만 존재한다.
5) 복잡한 SQL을 단계로 나누고 이름을 붙여 읽기 쉽게 만든다. 성능 향상 기능은 아니다.
6) 여러 개의 CTE를 쉼표로 연결할 수 있다.
7) 비재귀 WITH에서는 뒤의 CTE가 앞의 CTE를 참조하며, 그 반대는 허용되지 않는다.
8) JOIN과 함께 자주 사용된다.
9) 앞 단계 결과가 다음 단계 입력이 되는 분석에서 특히 유용하다.
10) 간단한 SQL은 굳이 CTE를 쓸 필요가 없다.